Day 19: Deploy a Linux Wazuh Agent

Theoretical Walkthrough of the Agent Deployment Procedure

SOC Internship - TriosCyber

Field	Details
Intern Name	Amrita Pathak
Date	22 September 2026
Internship Track	SOC (Security Operations Center)
Assigned Task	Install and enroll a Wazuh agent on a Linux endpoint; confirm it appears in agent management with active status. (Tools: Wazuh Agent, Linux VM)
Status of this Report	**Theoretical walkthrough of the deployment and enrollment procedure. Despite four days of sustained effort (18-22 September) and multiple deployment approaches, a working Wazuh manager could not be brought online due to insufficient hardware resources on the available lab machines, so this task could not be completed practically. This report demonstrates the correct procedure in full detail and will be re-submitted with real command output and screenshots once a manager is available.**

Table of Contents

Table of Contents 2
Key Terms 3
Important Note on This Report 3
Executive Summary 4
1. Objective 4
2. Prerequisite: A Running Wazuh Manager, and Why It Is Not Yet Available 4
3. Planned Procedure 5
3.1 Add the Wazuh package repository to the Linux endpoint 5
3.2 Install the agent, pointing it at the manager 5
3.3 Enable and start the agent service 6
3.4 Verify the service locally 6
3.5 Confirm enrollment from the Wazuh Dashboard 6
4. What Will Be Recorded as Evidence Once Executed 7
5. Conclusion 7

Key Terms

- **Wazuh Agent:** A lightweight program installed on an endpoint (server, workstation, VM) that collects security-relevant data (logs, file changes, running processes) and forwards it to the Wazuh manager for analysis.
- **Enrollment:** The process by which an agent registers itself with a Wazuh manager, exchanging cryptographic keys so the two can communicate over an encrypted, authenticated channel from that point on.
- **WAZUH_MANAGER:** An environment variable passed during agent installation that tells the agent the IP address or hostname of the Wazuh manager it should connect and enroll to.
- **ossec.conf:** The Wazuh agent's main configuration file (inherited from its OSSEC origins), where the manager address and other agent settings are defined; can be set at install time or edited afterward.
- **Agent Management / Agent List:** The section of the Wazuh Dashboard that lists every enrolled agent along with its status (active, disconnected, never connected) and last check-in time.
- **systemctl:** The Linux command used to start, stop, enable, and check the status of the wazuh-agent systemd service on the endpoint.
- **journalctl:** The Linux command used to view a systemd service's detailed logs; the planned tool for diagnosing the agent if it fails to start or fails to enroll.
- **Port 1514 / 1515:** The default TCP ports Wazuh agents use to communicate with the manager (1514 for encrypted agent-manager traffic, 1515 for the agent registration/enrollment service); a common enrollment failure point if blocked by a firewall.

Important Note on This Report

This is a theoretical walkthrough, not a record of a completed deployment. No Linux agent was installed or enrolled in this session, because no Wazuh manager is currently running to enroll against. The reason, and the four days of work behind it, are explained in Section 2 below.

Executive Summary

1. Objective

Install the Wazuh agent package on a Linux endpoint (a VM separate from the Wazuh manager), configure it to enroll with the manager, start the agent service, and confirm from the Wazuh Dashboard's agent management view that the endpoint shows an active status.

2. Prerequisite: A Running Wazuh Manager, and Why It Is Not Yet Available

Agent enrollment is fundamentally a client-server handshake: the agent needs a reachable manager IP address on the network to register against and to send data to afterward. Over four days of effort, that manager could not be brought online, for the following documented reasons:

- Attempts 1-6 (18-21 September): the Wazuh all-in-one installer was run repeatedly inside a VMware Workstation VM (Ubuntu Server 24.04, 6GB RAM, 2 vCPU) on an 8GB-RAM host. Certificate generation and the wazuh-indexer package install completed successfully every time, but the indexer service consistently failed to start, timing out after consuming several minutes of real CPU time without becoming ready.

- Diagnostic evidence gathered across these attempts (systemd logs showing a start-operation timeout after 5+ minutes of active CPU consumption, and kernel soft-lockup warnings showing a CPU core unresponsive for 20+ seconds) pointed to a specific root cause: OpenSearch (which the Wazuh indexer runs on) auto-sizes its Java heap based on available system RAM, and on a 6GB VM sharing an 8GB host with Windows and VMware itself, this default sizing created severe memory pressure and CPU scheduling contention rather than a simple slowdown.

- 22 September: based on that diagnosis, work began on a corrective fix, migrating off virtualization entirely onto a native, dual-boot Ubuntu Server installation. This required a different physical machine than the original lab laptop, since a dual-boot install partitions and repartitions a disk at the operating-system level, an operation that is very low-risk when done carefully but is not fully reversible if interrupted. The only machine with adequate free space for this was a shared family desktop, used daily by another household member for his own ongoing work, not a personal or disposable test machine.

  Before touching any partition, all existing data on the target drive was backed up as a precaution; however, the full contents of that drive could not be copied elsewhere in the time available, meaning a complete, verified backup of everything on the machine could not be guaranteed. Proceeding with an irreversible disk-level change to a shared machine, on data that could not be fully backed up, without another person's direct sign-off on that risk, was assessed as an unacceptable risk to take unilaterally, and the decision was made to pause the migration rather than proceed. This is treated here as a deliberate risk-management decision, consistent with standard change-control practice

of not applying an irreversible change to a shared system without a complete rollback path, rather than a shortcut.

The steps below therefore assume a manager is reachable at a known IP address, represented as <MANAGER_IP> throughout, and will be executed exactly as written once that manager exists.

3. Planned Procedure

3.1 Add the Wazuh package repository to the Linux endpoint

Before the agent package can be installed with apt, the Linux endpoint needs to trust and know about Wazuh's own package repository, which is not part of Ubuntu's default sources. This is a one-time setup per machine and involves three things: installing two small prerequisite tools, importing Wazuh's signing key so apt can verify the packages are genuine, and adding the repository's address to apt's list of sources.

```
sudo apt-get install gnupg apt-transport-https
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | gpg --no-default-keyring --
keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg --import
sudo chmod 644 /usr/share/keyrings/wazuh.gpg
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg]
https://packages.wazuh.com/4.x/apt/ stable main" | sudo tee -a
/etc/apt/sources.list.d/wazuh.list
sudo apt-get update
```

gnupg is the tool that handles the cryptographic signature verification; apt-transport-https allows apt to fetch packages over an encrypted connection. The curl command downloads Wazuh's public signing key and imports it into a dedicated keyring file (rather than the system's default keyring, which is the current best practice apt recommends). The chmod command makes that keyring file readable by the apt process. The echo/tee command writes one line into a new file under /etc/apt/sources.list.d/, telling apt where to find Wazuh's packages and which key to trust them against. The final apt-get update command re-scans all configured repositories, including the newly added one, so the wazuh-agent package becomes visible to apt in the next step.

3.2 Install the agent, pointing it at the manager

```
sudo WAZUH_MANAGER="<MANAGER_IP>" apt-get install wazuh-agent -y
```

This single command does two things at once: it installs the wazuh-agent package itself, and, because the WAZUH_MANAGER environment variable is set immediately before the command, the package's post-install script reads that variable and writes the manager's IP address directly into the agent's configuration file (ossec.conf) automatically. This means no manual text-editing

of a configuration file is required for a straightforward enrollment; the manager address is set correctly the moment the package finishes installing. The -y flag simply answers 'yes' automatically to apt's installation confirmation prompt, so the command can run unattended.

3.3 Enable and start the agent service

Installing the package does not automatically start it or configure it to start on future reboots; those are two separate, deliberate steps.

```
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

daemon-reload tells systemd to re-read its service definitions, which is good practice immediately after a new package registers a new service. enable configures wazuh-agent to start automatically on every future boot of the machine, so the agent does not need to be manually restarted after a reboot. start actually launches the service right now, in the current session.

3.4 Verify the service locally

```
sudo systemctl status wazuh-agent
```

A healthy result shows the line Active: active (running) along with the process ID and how long it has been running. This is the local, endpoint-side confirmation that the agent process itself started without error - it does not yet confirm the manager has accepted the enrollment, which is a separate check in Section 3.5.

If the status instead shows failed or inactive, the plan is to immediately pull the service's detailed log rather than treat a failed status as an unexplained dead end, the same evidence-first approach that successfully diagnosed the indexer failures in the earlier installation attempts:

```
sudo journalctl -u wazuh-agent --no-pager | tail -50
```

Common causes a log review would be expected to surface include: the manager IP being unreachable (wrong IP, or the two machines on different network segments), the manager's enrollment port (1515) being closed or blocked by a firewall on either side, or a leftover/corrupted client.keys file from a previous failed enrollment attempt, which typically requires deleting that file and re-registering.

3.5 Confirm enrollment from the Wazuh Dashboard

The local service running is necessary but not sufficient; the actual Day 19 requirement is that the endpoint shows up, and shows as active, on the manager's side. This is confirmed by logging into the Wazuh Dashboard (in a browser, at the manager's address), navigating to the Agents section (sometimes labeled Agent Management), and locating the new endpoint by the hostname of the Linux VM. A successful enrollment shows the agent listed with a status of Active (shown with a green indicator in the dashboard) and a Last Keep Alive timestamp that updates every roughly ten seconds, confirming live, ongoing communication rather than a one-time registration. This is the specific screen that will be captured as a screenshot once a manager is available, since it is the direct evidence the Day 19 task asks for.

4. What Will Be Recorded as Evidence Once Executed

- Full terminal output of the repository setup, agent install, and systemctl commands above.
- Output of systemctl status wazuh-agent showing active (running).
- A screenshot of the Wazuh Dashboard's Agent Management view showing the endpoint listed with Active status and a current Last Keep Alive timestamp.
- Any journalctl-based troubleshooting actually needed, along with the specific fix applied, if the agent does not connect on the first attempt.

5. Conclusion

Linux agent deployment was not practically performed in this session, and this report does not claim otherwise. Across four days, multiple documented approaches (repeated VM-based installation attempts, and the beginning of a dual-boot migration) were made to bring a Wazuh manager online, all constrained by the same underlying issue: insufficient hardware resources on the available lab machines for the virtualization-heavy approaches, and, for the resource-appropriate dual-boot alternative, the responsible decision not to complete an irreversible disk-level change on a shared, not-fully-backed-up household machine without another person's direct sign-off.

The procedure above is technically correct and verified against Wazuh's own official documentation, and demonstrates the understanding the Day 19 task is assessing. It will be executed for real, with genuine command output and a genuine dashboard screenshot, as soon as a Wazuh manager is available to enroll against.